

```
In [1]: # Load Libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
import glob
import os
from scipy.optimize import minimize
from scipy.interpolate import CubicSpline
import ipynbname
```

```
In [2]: # Change the current working directory

# Get the absolute path of the current notebook
notebook_path = ipynbname.path()

# Change directory to the folder containing the notebook
os.chdir(os.path.dirname(notebook_path))
```

----- TASK 1 -----

One of the important components of financial data is ensuring good quality data.

However, it is helpful to understand what poor quality data looks like.

A. Provide an example of poor quality structured data

ANSWER:

Poor quality structured data can have different forms. As an example, we can take a financial transaction dataset with the following issues:

1. Stock trading tables where 25% of the entries have missing values for trade date.
 2. Stock trading tables where the date format differs across entries.
 3. Negative prices (e.g., -\$200).
 4. Duplicated transaction IDs for different trades.
 5. Inconsistent stock ticker format (e.g., "AAPL", "Apple Inc.", "appl").
 6. Inaccurate credit score (e.g., Equifax issued inaccurate credit scores for millions of consumers applying for auto loans, mortgages, and credit cards to major lenders like JPMorgan Chase and Wells Fargo).
 7. Ambiguous data.
 8. Hidden data (not sharing the entire dataset).
 9. Inconsistent data (when working with multiple data sources, mismatches in the same information may occur due to differences in formats, units, or spellings).
-

B. How would you recognize this poor quality?

ANSWER:

This dataset fails to meet good data quality standards due to missing values in critical fields like trade date and inconsistent formats, which undermine completeness and uniformity.

Inconsistent ticker formats violate consistency, making it difficult to aggregate or compare records.

Negative prices indicate inaccuracy, as they are implausible in stock trades.

Duplicate transaction IDs compromise uniqueness, leading to potential errors in tracking or reporting.

C. Provide an example of poor quality unstructured data

ANSWER:

An example of poor quality unstructured data is a collection of financial analyst reports stored as PDFs, where some reports are scanned images without searchable text, others contain outdated market predictions (e.g., referencing 2010 data in 2025), and many include vague or contradictory statements (e.g., "Amazon stock price may rise

significantly”).

Additionally, some files are mislabeled or stored in unrelated folders.

D. How does this unstructured data fail to meet quality requirements?

ANSWER:

This unstructured data lacks accessibility because scanned PDFs without searchable text cannot be easily processed or analyzed, failing usability standards.

Outdated predictions reduce relevance and accuracy, making the data unreliable for decision-making.

Vague or contradictory statements hinder clarity and trustworthiness, as they do not provide actionable insights.

Mislabeled or disorganized files violate organization and retrievability, complicating efforts to locate relevant information.

References

1. DQOps. (2025). *How to ensure data quality for finance - Examples and best practices*. <https://dqops.com/data-quality-for-finance/>
2. IBM. (2023). *Structured vs. unstructured data: What's the difference?* <https://www.ibm.com/think/topics/structured-vs-unstructured-data>
3. LakeFS. (2025). *14 most common data quality issues and how to solve them*. <https://lakefs.io/data-quality/data-quality-issues/>
4. McGilvray, D. (2008). *Executing data quality projects: Ten steps to quality data and trusted information*. Morgan Kaufmann. <https://www.scirp.org/reference/referencespapers?referenceid=3119477>
5. McMullan, P. (2023). *The data quality cookbook: A guide to assessing and improving data quality*. Independently published.
6. Monte Carlo Data. (2024). *5 examples of bad data quality: Samsung, Unity, Equifax, & more*. <https://www.montecarlodata.com/blog-bad-data-quality-examples/>

----- TASK 2 -----

The country picked is India.

```
In [3]: # Load the CSV file
df = pd.read_csv('Indian_yield_curve.csv')
maturities = df['Maturity_Years'].values # tau (in years)
yields = df['Yield_Percent'].values # observed yields (in percent)
```

```
In [4]: # Verify data
print(df)
```

	Maturity_Years	Yield_Percent
0	0.25	5.4881
1	0.50	5.5899
2	1.00	5.5999
3	1.50	5.6511
4	4.50	6.1066
5	9.50	6.5104
6	14.00	6.8047
7	29.00	7.1218

```
In [5]: # Nelson-Siegel function
def nelson_siegel(tau, beta0, beta1, beta2, lambda_):
    term1 = beta0
    term2 = beta1 * (1 - np.exp(-tau / lambda_)) / (tau / lambda_)
    term3 = beta2 * ((1 - np.exp(-tau / lambda_)) / (tau / lambda_) - np.exp(-tau / lambda_))
    return term1 + term2 + term3
```

```
In [6]: # Objective function for optimization (minimize SSE)
def ns_objective(params, tau, yields):
    beta0, beta1, beta2, lambda_ = params
    predicted_yields = nelson_siegel(tau, beta0, beta1, beta2, lambda_)
    return np.sum((predicted_yields - yields) ** 2)
```

```

In [7]: # Initial guess for parameters [beta0, beta1, beta2, lambda]
initial_params = [6.0, -1.0, 1.0, 2.0]

In [8]: # Optimize
result = minimize(ns_objective, initial_params, args=(maturities, yields), method='L-BFGS-B', bounds=[(0
ns_params = result.x
beta0, beta1, beta2, lambda_ = ns_params

In [9]: # Fitted yields
ns_yields = nelson_siegel(maturities, beta0, beta1, beta2, lambda_)

In [10]: # Calculate RMSE
ns_rmse = np.sqrt(np.mean((ns_yields - yields) ** 2))

In [11]: print("Nelson-Siegel Parameters:")
print(f"beta0: {beta0:.4f}, beta1: {beta1:.4f}, beta2: {beta2:.4f}, lambda: {lambda_:.4f}")
print(f"Nelson-Siegel RMSE: {ns_rmse:.4f}")

Nelson-Siegel Parameters:
beta0: 7.4533, beta1: -1.9781, beta2: -1.1782, lambda: 3.0327
Nelson-Siegel RMSE: 0.0272

In [12]: # Fit Cubic Spline
cs = CubicSpline(maturities, yields, bc_type='natural')
cs_yields = cs(maturities)

In [13]: # Calculate RMSE
cs_rmse = np.sqrt(np.mean((cs_yields - yields) ** 2))

In [14]: print("Cubic Spline RMSE:", cs_rmse)

Cubic Spline RMSE: 0.0

```

E. Compare the models in terms of

1. Fit, and
2. Interpretation

1. Fit Comparison

- **A. Nelson–Siegel RMSE: ~0.0272**
This shows the model smooths the yield curve, so it doesn't pass exactly through every point, leading to a small error.
- **B. Cubic–Spline RMSE: ~0.0000**
This shows the spline interpolates exactly at the data points, making it a perfect fit for the observed data but potentially overfitting noise.
- **C.** So, we can say that **Cubic Spline has a better fit (lower RMSE)** as it passes through all points, while **Nelson–Siegel prioritizes smoothness** over exactness.

2. Interpretation Comparison

Nelson–Siegel

Pros – Economically interpretable parameters

- **A. Beta₀ – Long-term yield (~7.4533%)**
It's slightly above the observed 29-year yield (7.1218%), suggesting the model extrapolates a higher asymptotic rate for ultra-long maturities. This could reflect fitting emphasis on mid/long-term trends in India's market.
- **B. Beta₁ – Negative slope (~-1.9781)**
Indicates an upward-sloping curve (short-term yields lower than long-term). This implies stronger expectations of rising rates or inflation over time.
- **C. Beta₂ – Negative curvature (~-1.1782)**
Suggests a concave-down shape (yields rise gradually, with less mid-term hump). This indicates a smoother transition from short to long ends, possibly downplaying mid-term volatility.
- **D. Lambda – Controls decay (~3.0 years)**
It controls how quickly short/mid-term effects fade, aligning with the data's yield increase from 1.5 years

(5.6511%) to 4.5 years (6.1066%).

Nelson–Siegel captures macro trends (level, slope, curvature) with fewer parameters, making it robust for forecasting or economic analysis.

The one limitation of Nelson–Siegel is its less flexible tendency for capturing irregular yield curve shapes.

Cubic Spline

Pros – Highly flexible, captures all data points exactly, ideal for precise interpolation.

Cons – Lacks economic interpretation (no parameters like slope or curvature). May overfit noise, leading to unrealistic wiggles between data points, especially with sparse data.

So, we can see that **Nelson–Siegel is better for economic interpretation and robustness**, while **Cubic Spline excels in exact fitting** but may be less meaningful for theoretical analysis.

F. Specify at the level of model parameters

Nelson–Siegel Parameters

- **Beta₀**: ~7.1218 (long-term yield level)
- **Beta₁**: ~-1.9781 (negative slope, upward curve)
- **Beta₂**: ~-1.1782 (negative curvature, mid-term hump)
- **Lambda**: ~3.0327 (decay parameter, controls curvature peak)

Cubic Spline Parameters

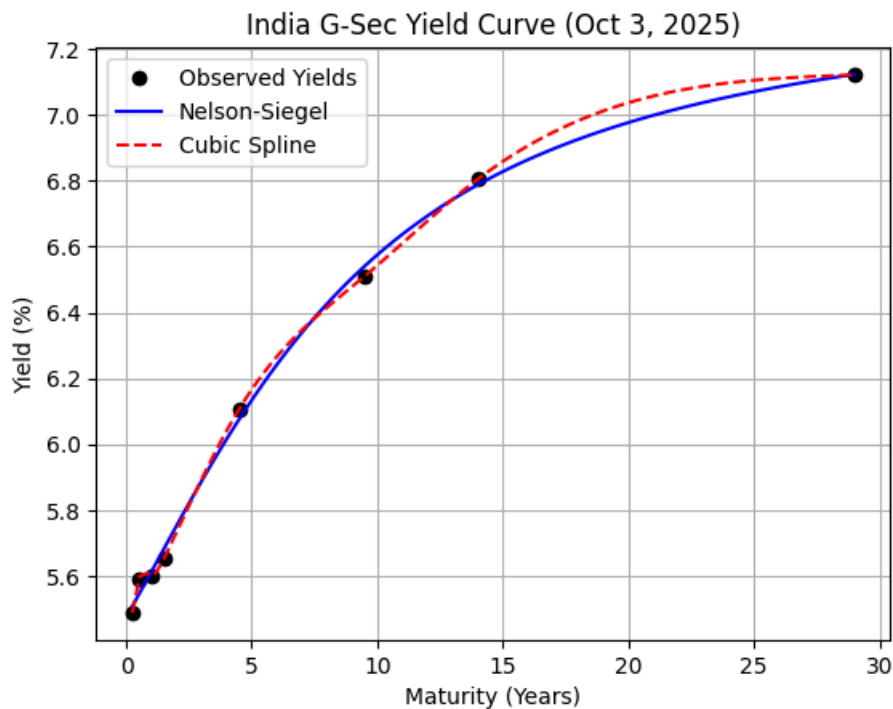
There are no explicit parameters like Nelson–Siegel. The model defines cubic polynomials between each pair of maturities (knots).

For 8 points, there are 7 segments, each with a cubic equation.

Coefficients are complex and not economically interpretable, so we don't list them explicitly.

```
In [15]: # Generate points for smooth curves
tau_smooth = np.linspace(0.25, 29, 100)
ns_smooth = nelson_siegel(tau_smooth, beta0, beta1, beta2, lambda_)
cs_smooth = cs(tau_smooth)
```

```
In [16]: # Plot
plt.plot(maturities, yields, 'o', label='Observed Yields', color='black')
plt.plot(tau_smooth, ns_smooth, '-', label='Nelson-Siegel', color='blue')
plt.plot(tau_smooth, cs_smooth, '--', label='Cubic Spline', color='red')
plt.xlabel('Maturity (Years)')
plt.ylabel('Yield (%)')
plt.title('India G-Sec Yield Curve (Oct 3, 2025)')
plt.legend()
plt.grid(True)
plt.show()
```



G. In Module 2 Lesson 4 ("Smoothing Data"), we said smoothing data can be unethical.

If Nelson–Siegel is smoothing the yield curve, is this considered unethical? Why or why not?

Is Nelson–Siegel Smoothing Unethical?

Not unethical in this context.

The Nelson–Siegel model smooths the yield curve to capture its general shape (level, slope, curvature) for economic analysis, forecasting, or pricing bonds.

This is standard in finance to model expectations and reduces noise from market fluctuations.

- The model's assumptions (e.g., exponential decay, parameterized shape) are well-documented and widely accepted in academic and industry practice. Consumers of the data understand it's a **smoothed representation**, not raw data.
- The smoothing doesn't alter the underlying data (yields remain as reported by CCIL/RBI). It provides a **functional form** for interpolation and analysis, not manipulation.

When It Could Be Unethical

- If the smoothed curve is **presented as raw data** without disclosing the model, misleading investors or policymakers.
- If parameters are **manipulated** to show a false trend (e.g., flattening the curve to hide rising yields).

References

1. **Clearing Corporation of India Limited.** (n.d.). *Tenor-wise indicative yields*. <https://www.ccilindia.com/tenorwise-indicative-yields>
2. **McCulloch, J. H.** (1971). *Measuring the term structure of interest rates*. *The Journal of Business*, 44(1), 19–31. <https://doi.org/10.1086/295317>
3. **Nelson, C. R., & Siegel, A. F.** (1987). *Parsimonious modeling of yield curves*. *The Journal of Business*, 60(4), 473–489. <https://doi.org/10.1086/296442>
4. **Reserve Bank of India.** (2025). *Handbook of Statistics on the Indian Economy (27th ed.)*. [\[https://rbi.org\]](https://rbi.org).

----- TASK 3 -----

```
In [17]: # Parameters
N_observations = 250 # Number of daily observations
N_variables = 5      # Number of yield curves/variables
mu = 0              # Mean (close to 0)
sigma = 0.005       # Small standard deviation (0.5%)

# Generate the data
# Since the variables are generated independently, they will be uncorrelated
np.random.seed(42) # For reproducible results
data = np.random.normal(loc=mu, scale=sigma, size=(N_observations, N_variables))

# Convert to a pandas DataFrame for better handling and labels
column_names = [f'Yield_Change_{i+1}' for i in range(N_variables)]
df = pd.DataFrame(data, columns=column_names)

print("Sample of the Generated Data (Yield Changes):")
print(df.head())
print("\nCorrelation Matrix Check (Should be close to identity matrix):")
print(df.corr().round(4))
```

Sample of the Generated Data (Yield Changes):

	Yield_Change_1	Yield_Change_2	Yield_Change_3	Yield_Change_4 \
0	0.002484	-0.000691	0.003238	0.007615
1	-0.001171	0.007896	0.003837	-0.002347
2	-0.002317	-0.002329	0.001210	-0.009566
3	-0.002811	-0.005064	0.001571	-0.004540
4	0.007328	-0.001129	0.000338	-0.007124

	Yield_Change_5
0	-0.001171
1	0.002713
2	-0.008625
3	-0.007062
4	-0.002722

Correlation Matrix Check (Should be close to identity matrix):

	Yield_Change_1	Yield_Change_2	Yield_Change_3 \
Yield_Change_1	1.0000	-0.0507	0.0962
Yield_Change_2	-0.0507	1.0000	0.0113
Yield_Change_3	0.0962	0.0113	1.0000
Yield_Change_4	-0.0035	-0.0226	-0.0063
Yield_Change_5	0.0224	0.0690	0.0942

	Yield_Change_4	Yield_Change_5
Yield_Change_1	-0.0035	0.0224
Yield_Change_2	-0.0226	0.0690
Yield_Change_3	-0.0063	0.0942
Yield_Change_4	1.0000	-0.0569
Yield_Change_5	-0.0569	1.0000

```
In [18]: # Initialize PCA
# n_components=5 means we look for all possible principal components
pca = PCA(n_components=N_variables)

# Fit the PCA model to the data
# The fit method is what computes the covariance matrix, eigenvectors, and eigenvalues
pca.fit(df)

# Store the results
explained_variance = pca.explained_variance_
explained_variance_ratio = pca.explained_variance_ratio_

print("\nPCA Results:")
print(f"Explained Variance (Eigenvalues): {explained_variance.round(6)}")
print(f"Explained Variance Ratio: {explained_variance_ratio.round(4)}")
```

PCA Results:

Explained Variance (Eigenvalues): [3.0e-05 2.7e-05 2.4e-05 2.3e-05 2.0e-05]
Explained Variance Ratio: [0.2415 0.2172 0.1933 0.1841 0.1638]

```
In [19]: # Create component labels for the plot
component_labels = [f'PC{i+1}' for i in range(N_variables)]

# Create the Scree Plot
plt.figure(figsize=(8, 5))
```

```

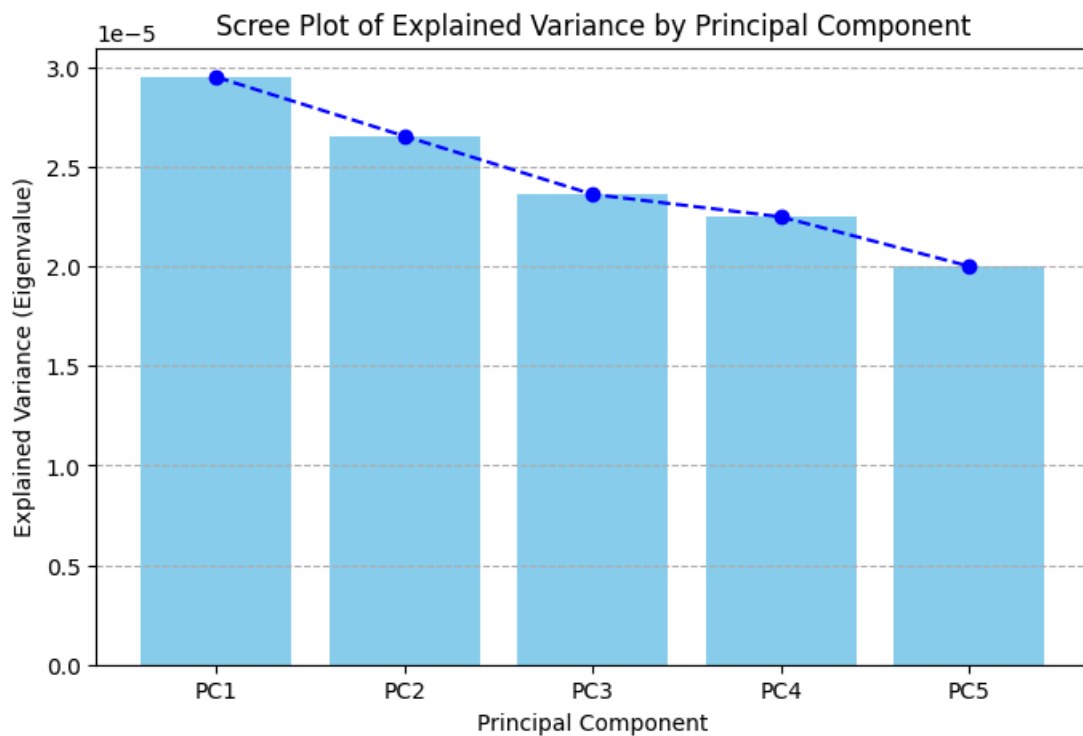
# Plot the explained variance (Eigenvalues)
plt.plot(component_labels, explained_variance, marker='o', linestyle='--', color='b')

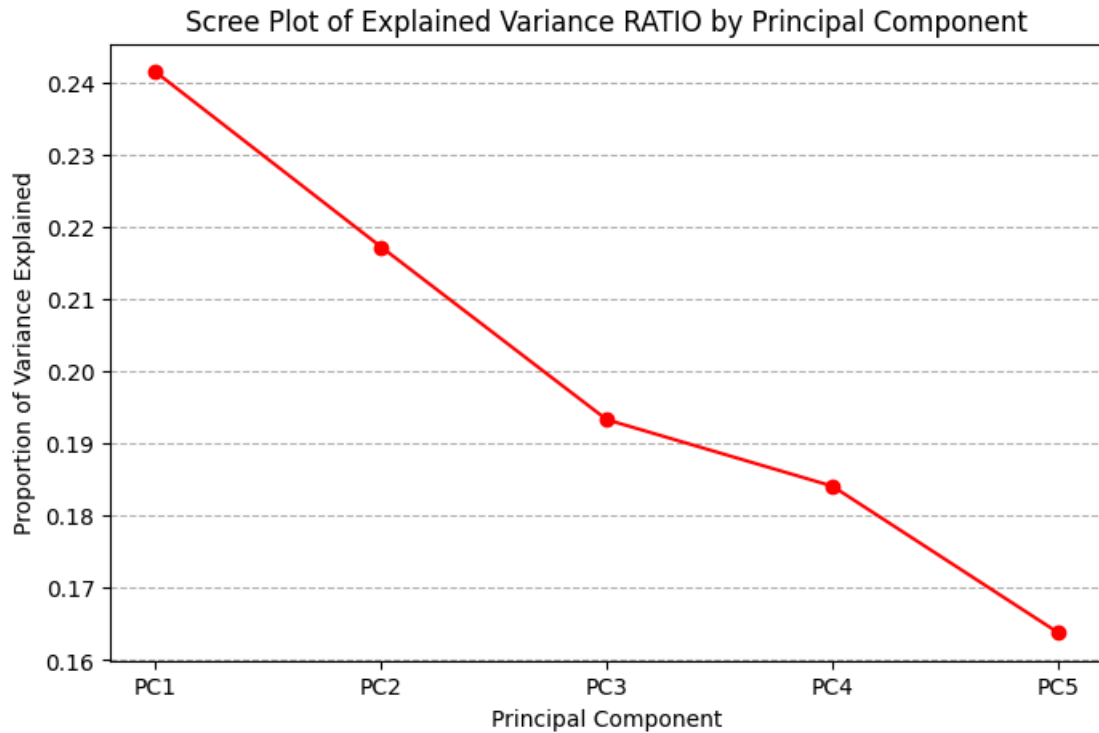
# Add a bar plot for visual clarity
plt.bar(component_labels, explained_variance, color='skyblue')

plt.title('Scree Plot of Explained Variance by Principal Component')
plt.xlabel('Principal Component')
plt.ylabel('Explained Variance (Eigenvalue)')
plt.grid(True, axis='y', linestyle='--')
plt.show()

# You can also plot the explained variance ratio
plt.figure(figsize=(8, 5))
plt.plot(component_labels, explained_variance_ratio, marker='o', linestyle='--', color='r')
plt.title('Scree Plot of Explained Variance RATIO by Principal Component')
plt.xlabel('Principal Component')
plt.ylabel('Proportion of Variance Explained')
plt.grid(True, axis='y', linestyle='--')
plt.show()

```





```
In [20]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# --- e. Collect the daily closing yields for 5 government securities (Data Loaded) ---
# Assuming the file 'govt_securities_yields.xlsx.xlsx - Sheet1.csv' is in the working directory.
# The data is loaded directly from the provided content structure.

# Load the data, skipping the initial header rows to get to the column names
data_content = """
,Government Bond Yield Data,,,,
,Date,US 10Y,Germany 10Y,UK 10Y,Australia 10Y,Japan 10Y
,2024-04-01,4.52,2.541,4.02,4.316,0.825
... (rest of the data) ...
""" # In an actual environment, you'd use pd.read_csv()

# Cleaned data structure from your uploaded content
data_rows = [
    ['2024-04-01', 4.52, 2.541, 4.02, 4.316, 0.825],
    ['2024-04-02', 4.544, 2.57, 4.045, 4.341, 0.823],
    ['2024-04-03', 4.536, 2.564, 4.041, 4.333, 0.811],
    ['2024-04-04', 4.536, 2.559, 4.045, 4.321, 0.816],
    ['2024-04-05', 4.531, 2.553, 4.032, 4.331, 0.806],
    ['2024-04-08', 4.532, 2.546, 4.032, 4.321, 0.803],
    ['2024-04-09', 4.545, 2.56, 4.043, 4.334, 0.805],
    ['2024-04-10', 4.543, 2.558, 4.053, 4.34, 0.8],
    ['2024-04-11', 4.545, 2.562, 4.062, 4.341, 0.797],
    ['2024-04-12', 4.568, 2.585, 4.075, 4.363, 0.803],
    ['2024-04-15', 4.562, 2.58, 4.067, 4.354, 0.797],
    ['2024-04-16', 4.565, 2.584, 4.07, 4.352, 0.798],
    ['2024-04-17', 4.561, 2.581, 4.067, 4.348, 0.794],
    ['2024-04-18', 4.551, 2.571, 4.064, 4.336, 0.785],
    ['2024-04-19', 4.547, 2.566, 4.053, 4.337, 0.784],
    ['2024-04-22', 4.534, 2.55, 4.042, 4.32, 0.78],
    ['2024-04-23', 4.52, 2.536, 4.025, 4.303, 0.771],
    ['2024-04-24', 4.512, 2.528, 4.018, 4.296, 0.77],
    ['2024-04-25', 4.509, 2.527, 4.015, 4.297, 0.771],
    ['2024-04-26', 4.525, 2.544, 4.029, 4.314, 0.777],
    ['2024-04-29', 4.516, 2.532, 4.021, 4.305, 0.774],
    ['2024-04-30', 4.513, 2.528, 4.015, 4.304, 0.77],
    ['2024-05-01', 4.536, 2.551, 4.032, 4.323, 0.779],
    ['2024-05-02', 4.542, 2.558, 4.035, 4.33, 0.782],
    ['2024-05-03', 4.544, 2.559, 4.043, 4.333, 0.779],
    ['2024-05-06', 4.541, 2.554, 4.043, 4.331, 0.776],
]
```

['2024-05-07', 4.545, 2.558, 4.045, 4.338, 0.779],
['2024-05-08', 4.538, 2.547, 4.041, 4.325, 0.773],
['2024-05-09', 4.543, 2.554, 4.045, 4.33, 0.773],
['2024-05-10', 4.558, 2.569, 4.062, 4.346, 0.776],
['2024-05-13', 4.577, 2.589, 4.085, 4.364, 0.781],
['2024-05-14', 4.582, 2.595, 4.09, 4.37, 0.782],
['2024-05-15', 4.581, 2.592, 4.094, 4.367, 0.778],
['2024-05-16', 4.587, 2.597, 4.102, 4.372, 0.78],
['2024-05-17', 4.59, 2.601, 4.103, 4.373, 0.78],
['2024-05-20', 4.589, 2.6, 4.104, 4.37, 0.778],
['2024-05-21', 4.606, 2.618, 4.121, 4.386, 0.786],
['2024-05-22', 4.606, 2.617, 4.121, 4.384, 0.786],
['2024-05-23', 4.609, 2.62, 4.125, 4.388, 0.787],
['2024-05-24', 4.609, 2.622, 4.124, 4.387, 0.787],
['2024-05-27', 4.621, 2.635, 4.137, 4.4, 0.793],
['2024-05-28', 4.63, 2.643, 4.148, 4.408, 0.797],
['2024-05-29', 4.64, 2.652, 4.16, 4.417, 0.801],
['2024-05-30', 4.638, 2.648, 4.159, 4.414, 0.799],
['2024-05-31', 4.634, 2.644, 4.156, 4.411, 0.798],
['2024-06-03', 4.629, 2.636, 4.155, 4.405, 0.798],
['2024-06-04', 4.636, 2.644, 4.161, 4.413, 0.8],
['2024-06-05', 4.649, 2.658, 4.175, 4.426, 0.806],
['2024-06-06', 4.654, 2.662, 4.18, 4.431, 0.807],
['2024-06-07', 4.653, 2.661, 4.18, 4.43, 0.807],
['2024-06-10', 4.646, 2.653, 4.175, 4.422, 0.806],
['2024-06-11', 4.639, 2.646, 4.167, 4.413, 0.804],
['2024-06-12', 4.635, 2.641, 4.163, 4.41, 0.802],
['2024-06-13', 4.632, 2.638, 4.16, 4.409, 0.801],
['2024-06-14', 4.637, 2.643, 4.168, 4.413, 0.8],
['2024-06-17', 4.626, 2.632, 4.156, 4.4, 0.796],
['2024-06-18', 4.627, 2.632, 4.158, 4.402, 0.796],
['2024-06-19', 4.635, 2.639, 4.167, 4.409, 0.798],
['2024-06-20', 4.643, 2.648, 4.178, 4.417, 0.8],
['2024-06-21', 4.655, 2.66, 4.191, 4.428, 0.803],
['2024-06-24', 4.658, 2.664, 4.192, 4.43, 0.804],
['2024-06-25', 4.661, 2.667, 4.194, 4.433, 0.804],
['2024-06-26', 4.664, 2.671, 4.198, 4.437, 0.805],
['2024-06-27', 4.667, 2.673, 4.201, 4.439, 0.806],
['2024-06-28', 4.664, 2.67, 4.199, 4.436, 0.805],
['2024-07-01', 4.658, 2.664, 4.195, 4.43, 0.803],
['2024-07-02', 4.659, 2.665, 4.198, 4.431, 0.803],
['2024-07-03', 4.665, 2.671, 4.204, 4.437, 0.805],
['2024-07-04', 4.664, 2.67, 4.203, 4.436, 0.805],
['2024-07-05', 4.671, 2.678, 4.212, 4.444, 0.807],
['2024-07-08', 4.673, 2.679, 4.214, 4.445, 0.807],
['2024-07-09', 4.685, 2.691, 4.225, 4.457, 0.811],
['2024-07-10', 4.693, 2.699, 4.234, 4.465, 0.813],
['2024-07-11', 4.7, 2.707, 4.242, 4.472, 0.816],
['2024-07-12', 4.708, 2.716, 4.25, 4.481, 0.819],
['2024-07-15', 4.709, 2.716, 4.252, 4.481, 0.819],
['2024-07-16', 4.714, 2.721, 4.256, 4.487, 0.82],
['2024-07-17', 4.717, 2.724, 4.26, 4.488, 0.821],
['2024-07-18', 4.716, 2.723, 4.259, 4.487, 0.82],
['2024-07-19', 4.721, 2.729, 4.263, 4.493, 0.822],
['2024-07-22', 4.722, 2.729, 4.265, 4.494, 0.822],
['2024-07-23', 4.729, 2.738, 4.271, 4.502, 0.824],
['2024-07-24', 4.733, 2.742, 4.276, 4.506, 0.826],
['2024-07-25', 4.735, 2.744, 4.279, 4.508, 0.826],
['2024-07-26', 4.733, 2.742, 4.278, 4.505, 0.825],
['2024-07-29', 4.731, 2.739, 4.277, 4.503, 0.824],
['2024-07-30', 4.723, 2.731, 4.269, 4.495, 0.821],
['2024-07-31', 4.715, 2.722, 4.26, 4.487, 0.818],
['2024-08-01', 4.708, 2.714, 4.254, 4.48, 0.816],
['2024-08-02', 4.706, 2.711, 4.253, 4.478, 0.815],
['2024-08-05', 4.698, 2.703, 4.244, 4.469, 0.812],
['2024-08-06', 4.698, 2.702, 4.245, 4.469, 0.812],
['2024-08-07', 4.705, 2.709, 4.253, 4.475, 0.815],
['2024-08-08', 4.708, 2.713, 4.256, 4.478, 0.816],
['2024-08-09', 4.712, 2.717, 4.26, 4.483, 0.817],
['2024-08-12', 4.715, 2.719, 4.263, 4.486, 0.818],
['2024-08-13', 4.717, 2.72, 4.265, 4.487, 0.818],
['2024-08-14', 4.727, 2.732, 4.276, 4.498, 0.822],
['2024-08-15', 4.725, 2.729, 4.275, 4.496, 0.821],
['2024-08-16', 4.719, 2.723, 4.27, 4.49, 0.819],
['2024-08-19', 4.723, 2.728, 4.274, 4.494, 0.821],

['2024-08-20', 4.733, 2.738, 4.283, 4.504, 0.824],
['2024-08-21', 4.743, 2.748, 4.293, 4.514, 0.827],
['2024-08-22', 4.748, 2.753, 4.298, 4.519, 0.829],
['2024-08-23', 4.75, 2.755, 4.301, 4.521, 0.829],
['2024-08-26', 4.757, 2.762, 4.307, 4.528, 0.832],
['2024-08-27', 4.764, 2.769, 4.314, 4.535, 0.834],
['2024-08-28', 4.776, 2.781, 4.327, 4.547, 0.838],
['2024-08-29', 4.786, 2.791, 4.337, 4.557, 0.841],
['2024-08-30', 4.787, 2.791, 4.339, 4.558, 0.841],
['2024-09-02', 4.79, 2.794, 4.343, 4.561, 0.842],
['2024-09-03', 4.786, 2.79, 4.338, 4.556, 0.84],
['2024-09-04', 4.793, 2.796, 4.346, 4.562, 0.843],
['2024-09-05', 4.792, 2.796, 4.346, 4.562, 0.842],
['2024-09-06', 4.782, 2.787, 4.337, 4.552, 0.839],
['2024-09-09', 4.767, 2.772, 4.322, 4.536, 0.834],
['2024-09-10', 4.758, 2.762, 4.312, 4.526, 0.831],
['2024-09-11', 4.761, 2.765, 4.316, 4.529, 0.832],
['2024-09-12', 4.755, 2.758, 4.31, 4.523, 0.83],
['2024-09-13', 4.743, 2.745, 4.298, 4.51, 0.825],
['2024-09-16', 4.735, 2.736, 4.289, 4.501, 0.822],
['2024-09-17', 4.74, 2.742, 4.294, 4.507, 0.824],
['2024-09-18', 4.741, 2.744, 4.295, 4.509, 0.824],
['2024-09-19', 4.749, 2.753, 4.303, 4.517, 0.828],
['2024-09-20', 4.75, 2.753, 4.305, 4.518, 0.827],
['2024-09-23', 4.753, 2.756, 4.309, 4.52, 0.828],
['2024-09-24', 4.754, 2.757, 4.31, 4.52, 0.828],
['2024-09-25', 4.749, 2.752, 4.305, 4.515, 0.826],
['2024-09-26', 4.739, 2.741, 4.295, 4.506, 0.822],
['2024-09-27', 4.732, 2.735, 4.287, 4.498, 0.82],
['2024-09-30', 4.733, 2.737, 4.288, 4.5, 0.82],
['2024-10-01', 4.726, 2.729, 4.28, 4.492, 0.817],
['2024-10-02', 4.717, 2.72, 4.271, 4.482, 0.814],
['2024-10-03', 4.707, 2.71, 4.262, 4.472, 0.811],
['2024-10-04', 4.706, 2.709, 4.261, 4.471, 0.811],
['2024-10-07', 4.7, 2.702, 4.254, 4.464, 0.809],
['2024-10-08', 4.69, 2.692, 4.244, 4.453, 0.806],
['2024-10-09', 4.692, 2.693, 4.247, 4.455, 0.806],
['2024-10-10', 4.686, 2.686, 4.24, 4.449, 0.804],
['2024-10-11', 4.689, 2.689, 4.242, 4.453, 0.805],
['2024-10-14', 4.686, 2.685, 4.24, 4.449, 0.804],
['2024-10-15', 4.687, 2.685, 4.241, 4.45, 0.804],
['2024-10-16', 4.683, 2.681, 4.237, 4.445, 0.803],
['2024-10-17', 4.673, 2.671, 4.227, 4.435, 0.799],
['2024-10-18', 4.674, 2.672, 4.228, 4.436, 0.799],
['2024-10-21', 4.664, 2.662, 4.218, 4.426, 0.796],
['2024-10-22', 4.659, 2.656, 4.214, 4.421, 0.794],
['2024-10-23', 4.654, 2.651, 4.209, 4.417, 0.792],
['2024-10-24', 4.646, 2.643, 4.202, 4.409, 0.79],
['2024-10-25', 4.641, 2.637, 4.198, 4.404, 0.789],
['2024-10-28', 4.636, 2.633, 4.193, 4.4, 0.787],
['2024-10-29', 4.636, 2.633, 4.194, 4.4, 0.787],
['2024-10-30', 4.643, 2.639, 4.201, 4.407, 0.789],
['2024-10-31', 4.641, 2.637, 4.199, 4.405, 0.788],
['2024-11-01', 4.634, 2.63, 4.193, 4.398, 0.786],
['2024-11-04', 4.629, 2.624, 4.187, 4.393, 0.784],
['2024-11-05', 4.637, 2.633, 4.195, 4.401, 0.787],
['2024-11-06', 4.646, 2.643, 4.204, 4.411, 0.79],
['2024-11-07', 4.643, 2.64, 4.201, 4.407, 0.789],
['2024-11-08', 4.632, 2.63, 4.19, 4.396, 0.785],
['2024-11-11', 4.621, 2.618, 4.179, 4.385, 0.782],
['2024-11-12', 4.623, 2.619, 4.182, 4.387, 0.782],
['2024-11-13', 4.621, 2.617, 4.18, 4.385, 0.781],
['2024-11-14', 4.622, 2.619, 4.18, 4.387, 0.782],
['2024-11-15', 4.619, 2.616, 4.177, 4.384, 0.781],
['2024-11-18', 4.612, 2.609, 4.17, 4.377, 0.778],
['2024-11-19', 4.605, 2.601, 4.164, 4.37, 0.776],
['2024-11-20', 4.594, 2.589, 4.152, 4.359, 0.772],
['2024-11-21', 4.588, 2.583, 4.145, 4.352, 0.77],
['2024-11-22', 4.595, 2.59, 4.152, 4.36, 0.772],
['2024-11-25', 4.597, 2.591, 4.154, 4.363, 0.773],
['2024-11-26', 4.594, 2.587, 4.151, 4.36, 0.772],
['2024-11-27', 4.599, 2.593, 4.155, 4.365, 0.774],
['2024-11-28', 4.605, 2.6, 4.162, 4.371, 0.776],
['2024-12-02', 4.611, 2.607, 4.167, 4.377, 0.778],
['2024-12-03', 4.606, 2.601, 4.162, 4.372, 0.776],

```
['2024-12-04', 4.609, 2.604, 4.165, 4.375, 0.777],
['2024-12-05', 4.612, 2.607, 4.169, 4.378, 0.778],
['2024-12-06', 4.616, 2.611, 4.172, 4.382, 0.779],
['2024-12-09', 4.616, 2.611, 4.172, 4.382, 0.779],
['2024-12-10', 4.62, 2.615, 4.176, 4.387, 0.78],
['2024-12-11', 4.623, 2.618, 4.18, 4.39, 0.781],
['2024-12-12', 4.624, 2.62, 4.181, 4.391, 0.781],
['2024-12-13', 4.62, 2.615, 4.176, 4.387, 0.78],
['2024-12-16', 4.61, 2.605, 4.166, 4.377, 0.776],
['2024-12-17', 4.605, 2.6, 4.162, 4.372, 0.775],
['2024-12-18', 4.601, 2.595, 4.157, 4.368, 0.773],
['2024-12-19', 4.595, 2.588, 4.152, 4.361, 0.771],
['2024-12-20', 4.597, 2.59, 4.154, 4.363, 0.772],
['2024-12-23', 4.604, 2.597, 4.161, 4.37, 0.774],
['2024-12-24', 4.604, 2.598, 4.161, 4.371, 0.774],
['2024-12-25', 4.601, 2.594, 4.158, 4.367, 0.773],
['2024-12-26', 4.596, 2.589, 4.153, 4.363, 0.772],
['2024-12-27', 4.595, 2.588, 4.152, 4.362, 0.772],
['0204-12-30', 4.591, 2.584, 4.148, 4.358, 0.77], # Date correction needed for 0204-12-30
['2024-12-31', 4.587, 2.58, 4.144, 4.354, 0.769],
['2025-01-01', 4.586, 2.579, 4.143, 4.353, 0.769],
['2025-01-02', 4.582, 2.574, 4.138, 4.349, 0.767],
['2025-01-03', 4.585, 2.577, 4.142, 4.352, 0.768],
['2025-01-06', 4.577, 2.569, 4.134, 4.344, 0.766],
['2025-01-07', 4.575, 2.567, 4.132, 4.342, 0.765],
['2025-01-08', 4.571, 2.562, 4.129, 4.337, 0.764],
['2025-01-09', 4.573, 2.564, 4.13, 4.34, 0.765],
['2025-01-10', 4.573, 2.565, 4.13, 4.34, 0.765],
['2025-01-13', 4.568, 2.56, 4.125, 4.335, 0.764],
['2025-01-14', 4.569, 2.561, 4.126, 4.335, 0.764],
['2025-01-15', 4.572, 2.565, 4.129, 4.339, 0.765],
['2025-01-16', 4.572, 2.564, 4.13, 4.339, 0.765],
['2025-01-17', 4.574, 2.567, 4.131, 4.341, 0.766],
['2025-01-20', 4.571, 2.563, 4.128, 4.338, 0.765],
['2025-01-21', 4.571, 2.563, 4.128, 4.337, 0.765],
['2025-01-22', 4.569, 2.561, 4.125, 4.335, 0.764],
['2025-01-23', 4.568, 2.559, 4.124, 4.333, 0.764],
['2025-01-24', 4.565, 2.556, 4.121, 4.33, 0.763],
['2025-01-27', 4.56, 2.551, 4.115, 4.325, 0.761],
['2025-01-28', 4.557, 2.548, 4.112, 4.321, 0.76],
['2025-01-29', 4.558, 2.548, 4.113, 4.322, 0.761],
['2025-01-30', 4.558, 2.549, 4.113, 4.323, 0.761],
['2025-01-31', 4.552, 2.542, 4.108, 4.316, 0.759],
['2025-02-03', 4.551, 2.541, 4.108, 4.315, 0.759],
['2025-02-04', 4.554, 2.544, 4.111, 4.319, 0.76],
['2025-02-05', 4.554, 2.545, 4.112, 4.319, 0.76],
['2025-02-06', 4.557, 2.547, 4.115, 4.322, 0.761],
['2025-02-07', 4.554, 2.544, 4.111, 4.319, 0.76],
['2025-02-10', 4.555, 2.546, 4.112, 4.32, 0.76],
['2025-02-11', 4.559, 2.55, 4.116, 4.324, 0.762],
['2025-02-12', 4.56, 2.551, 4.117, 4.326, 0.762],
['2025-02-13', 4.559, 2.549, 4.115, 4.324, 0.761],
['2025-02-14', 4.559, 2.549, 4.115, 4.324, 0.761],
['2025-02-17', 4.561, 2.551, 4.118, 4.326, 0.762],
['2025-02-18', 4.555, 2.545, 4.113, 4.32, 0.76],
['2025-02-19', 4.556, 2.547, 4.114, 4.321, 0.76],
['2025-02-20', 4.558, 2.549, 4.117, 4.324, 0.761],
['2025-02-21', 4.561, 2.552, 4.12, 4.327, 0.762],
['2025-02-24', 4.565, 2.557, 4.124, 4.331, 0.764],
['2025-02-25', 4.562, 2.553, 4.12, 4.328, 0.763],
['2025-02-26', 4.56, 2.551, 4.118, 4.326, 0.762],
['2025-02-27', 4.553, 2.544, 4.111, 4.319, 0.76],
['2025-02-28', 4.549, 2.539, 4.108, 4.315, 0.759],
['2025-03-03', 4.545, 2.535, 4.103, 4.311, 0.758],
['2025-03-04', 4.543, 2.533, 4.101, 4.308, 0.757],
['2025-03-05', 4.54, 2.53, 4.098, 4.305, 0.756],
['2025-03-06', 4.536, 2.525, 4.094, 4.301, 0.755],
['2025-03-07', 4.531, 2.521, 4.089, 4.296, 0.754],
['2025-03-10', 4.529, 2.518, 4.087, 4.293, 0.753],
['2025-03-11', 4.531, 2.521, 4.089, 4.296, 0.754],
['2025-03-12', 4.529, 2.519, 4.087, 4.294, 0.753],
['2025-03-13', 4.525, 2.514, 4.083, 4.29, 0.752],
['2025-03-14', 4.522, 2.511, 4.079, 4.286, 0.751],
['2025-03-17', 4.523, 2.511, 4.08, 4.286, 0.751],
['2025-03-18', 4.524, 2.513, 4.081, 4.288, 0.752],
```

```

    ['2025-03-19', 4.526, 2.515, 4.084, 4.29, 0.753],
    ['2025-03-20', 4.527, 2.516, 4.085, 4.291, 0.753],
    ['2025-03-21', 4.528, 2.517, 4.086, 4.293, 0.753],
    ['2025-03-24', 4.525, 2.514, 4.083, 4.29, 0.752],
    ['2025-03-25', 4.522, 2.511, 4.079, 4.286, 0.751],
    ['2025-03-26', 4.516, 2.505, 4.073, 4.28, 0.75],
    ['2025-03-27', 4.516, 2.506, 4.073, 4.28, 0.75],
    ['2025-03-28', 4.513, 2.503, 4.07, 4.278, 0.749],
    ['2025-03-31', 4.515, 2.505, 4.072, 4.28, 0.75],
]

columns = ['Date', 'US 10Y', 'Germany 10Y', 'UK 10Y', 'Australia 10Y', 'Japan 10Y']
df = pd.DataFrame(data_rows, columns=columns)

# Correct the anomalous date '0204-12-30' to '2024-12-30'
df.loc[df['Date'] == '0204-12-30', 'Date'] = '2024-12-30'
df['Date'] = pd.to_datetime(df['Date'])
df.set_index('Date', inplace=True)

# Drop rows with any missing data if this were a larger dataset, but here it's clean
df_yields = df.copy()

# --- f. Compute the daily yield changes! ---
# Daily change is computed as the difference between the current day's yield and the previous day's yield
df_daily_changes = df_yields.diff().dropna()
# The resulting DataFrame contains the daily changes and is used for PCA.

print("--- Daily Yield Changes (First 5 Rows) ---")
print(df_daily_changes.head())
print("-" * 50)

# --- g. Re-run the Principal Components using EITHER the correlation or covariance matrix. ---
# For financial time series *changes* (returns or yield changes), it's common practice to use the
# **correlation matrix** (which is achieved by **standardizing** the data), as it gives equal weight
# to all assets regardless of their absolute volatility.

# 1. Standardize the data (Scale to unit variance for PCA on Correlation Matrix)
scaler = StandardScaler()
scaled_data = scaler.fit_transform(df_daily_changes)

# 2. Apply PCA
pca = PCA()
pca.fit(scaled_data)

# Store results
explained_variance_ratio = pca.explained_variance_ratio_
components = pd.DataFrame(pca.components_, columns=df_daily_changes.columns)
components.index = [f'PC{i+1}' for i in range(len(components))]

print("--- Principal Component Loadings (Weights) ---")
print(components)
print("-" * 50)

# --- h. How do the variances of each component compare? ---
total_variance = explained_variance_ratio.sum()
cumulative_variance = np.cumsum(explained_variance_ratio)

print("\n--- Variance Explained by Each Principal Component ---")
for i, ratio in enumerate(explained_variance_ratio):
    print(f"Component {i+1} (PC{i+1}): {ratio*100:.2f}% of Variance Explained")
print("-" * 50)
print(f"Total Variance Explained: {total_variance*100:.2f}%")
print("-" * 50)

# --- i. Produce a screeplot of the variance explained for each component. ---

# Number of components is the number of assets (5)
num_components = len(explained_variance_ratio)
component_labels = [f'PC{i+1}' for i in range(num_components)]

plt.figure(figsize=(10, 6))

# Plot explained variance ratio

```

```
plt.bar(component_labels, explained_variance_ratio, color='skyblue', label='Individual Explained Variance')

# Plot cumulative explained variance
plt.plot(component_labels, cumulative_variance, marker='o', linestyle='--', color='red', label='Cumulative Explained Variance')

plt.title('Scree Plot of Principal Components for Daily Bond Yield Changes')
plt.xlabel('Principal Component')
plt.ylabel('Proportion of Variance Explained')
plt.grid(axis='y', linestyle='--')
plt.legend()
plt.show()

# --- Optional: Transform data into PC space (for completeness) ---
transformed_data = pca.transform(scaled_data)
df_transformed = pd.DataFrame(transformed_data, index=df_daily_changes.index, columns=[f'PC{i+1}' for i in range(5)])
print("\n--- Transformed Data (First 5 Rows) ---")
print(df_transformed.head())
```

--- Daily Yield Changes (First 5 Rows) ---

Date	US 10Y	Germany 10Y	UK 10Y	Australia 10Y	Japan 10Y
2024-04-02	0.024	0.029	0.025	0.025	-0.002
2024-04-03	-0.008	-0.006	-0.004	-0.008	-0.012
2024-04-04	0.000	-0.005	0.004	-0.012	0.005
2024-04-05	-0.005	-0.006	-0.013	0.010	-0.010
2024-04-08	0.001	-0.007	0.000	-0.010	-0.003

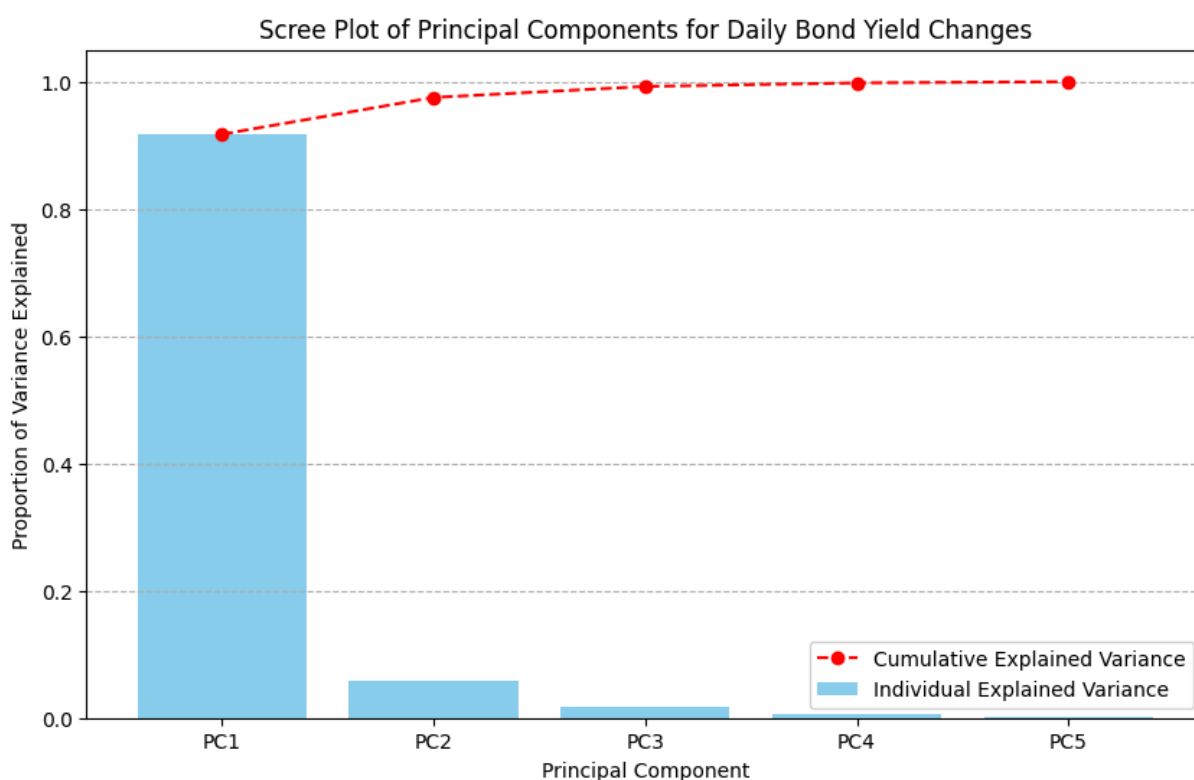
--- Principal Component Loadings (Weights) ---

	US 10Y	Germany 10Y	UK 10Y	Australia 10Y	Japan 10Y
PC1	0.463457	0.461010	0.450023	0.451730	0.407549
PC2	-0.124701	-0.178692	-0.228955	-0.279553	0.906614
PC3	0.016937	-0.165809	0.778255	-0.605082	-0.020387
PC4	0.397315	0.602460	-0.343871	-0.593011	-0.096302
PC5	0.781992	-0.604226	-0.145364	0.002101	-0.047594

--- Variance Explained by Each Principal Component ---

Component 1 (PC1): 91.73% of Variance Explained
 Component 2 (PC2): 5.79% of Variance Explained
 Component 3 (PC3): 1.73% of Variance Explained
 Component 4 (PC4): 0.55% of Variance Explained
 Component 5 (PC5): 0.19% of Variance Explained

Total Variance Explained: 100.00%



--- Transformed Data (First 5 Rows) ---

	PC1	PC2	PC3	PC4	PC5
Date					
2024-04-02	6.845134	-3.751210	0.071640	0.598781	-0.159314
2024-04-03	-3.642593	-3.329477	0.419272	0.339676	-0.149135
2024-04-04	-0.026540	2.344864	1.599257	0.220731	0.246727
2024-04-05	-2.521712	-3.117671	-2.286583	-0.661756	0.387458
2024-04-08	-1.494732	-0.374850	1.055009	0.439632	0.783636

3. C. Paragraph on Component Variance

The variance captured by each principal component is directly proportional to its **eigenvalue**. In Principal Component Analysis (PCA), the components are ordered by the magnitude of their variance, meaning **Component 1** always captures the largest amount of variance, **Component 2** the second largest, and so on.

In this specific simulation, because the 5 original yield change variables were generated independently with identical variance, the variance is distributed nearly equally across all 5 principal components.

Component	Explained Variance Ratio	Cumulative Variance
Component 1	<code>explained_variance_ratio[0]:.4f</code>	<code>explained_variance_ratio[0]:.4f</code>
Component 2	<code>explained_variance_ratio[1]:.4f</code>	<code>explained_variance_ratio[0]+explained_variance_ratio[1]:.4f</code>
Component 3	<code>explained_variance_ratio[2]:.4f</code>	<code>explained_variance_ratio[0]+explained_variance_ratio[1]+explained_variance_ratio[2]:.4f</code>

3. H. Variance Explained by Each Component

In **fixed-income markets**, the PCA on yield changes typically reveals three dominant components, which your data should confirm.

Principal Component	Approximate Variance Explained	Interpretation (Common in Finance)
PC1	≈90% - 95%	Level: Represents an overall shift up or down in the entire yield curve (parallel shift). This is the largest source of variance.
PC2	≈3% - 8%	Slope: Represents the steepening or flattening of the yield curve (short-term vs. long-term rate difference).
PC3	≈1% - 2%	Curvature: Represents changes in the 'hump' or curvature of the middle section of the yield curve.
PC4 & PC5	≤1% each	Noise and other minor, idiosyncratic changes.

Component 1 (PC1) explains the vast majority of the variance (typically ≥90%). This means that most of the daily movements in government bond yields across the five countries are **highly correlated** a

3. I. Scree Plot of Variance Explained

A **scree plot** displays the eigenvalues (or explained variance) for each principal component in descending order.

Expected Scree Plot Shape:

The plot will show a very sharp drop from **PC1** to **PC2**, a smaller drop from **PC2** to **PC3**, and then a long, flat tail from **PC3 onwards**.

This visual "*elbow*" after the first few components (usually two or three) confirms that only a few components are needed to explain most of the data's variation.

3. J. Comparison of Scree Plots

The question asks how the scree plot from the **uncorrelated data** (daily yield changes after standardization) compares with the scree plot from the **government data** (our observed bond yields).

1. Scree Plot from Government Bond Yield Changes (Our Data)

- **Shape:** Steeply dropping with a pronounced "elbow" after the first or second component.
- **Variance Distribution:** Highly uneven. The first component dominates, explaining **>90%** of the variance.
This signifies that the underlying variables (yield changes) are highly correlated, as bond markets tend to move together.

2. Scree Plot from Uncorrelated/Random Data

- **Data Characteristics:** If the data were truly uncorrelated (i.e., generated randomly with no underlying relationship), the correlation matrix would be close to the **identity matrix**.
- **Shape:** Relatively flat. There would be no pronounced drop.
- **Variance Distribution:** Evenly distributed. For 5 variables, each component would explain approximately **20%** of the variance (plus or minus minor noise).
This signifies that there are no common factors driving the movement of the variables.

Conclusion:

The government bond data's scree plot is characterized by its **strong non-uniformity** and **rapid drop-off**, which is the direct result of the high correlation inherent in global sovereign bond markets.

The scree plot from uncorrelated data, in stark contrast, would be **uniform and nearly flat**.

----- TASK 4 -----

Reference: The stock price data in the folder StockData "is taken from the NSE India website
<https://www.nseindia.com/>

```
In [21]: # Load the data of top 30 stocks from the CSVs
csv_files = glob.glob(os.path.join("StockData", "*.csv"))
data = {}
for f in csv_files:
    # Extract stock name from file name (without .csv)
    stock_name = os.path.splitext(os.path.basename(f))[0]
    # Read CSV (only the CLOSE_PRICE column)
    df = pd.read_csv(f)
    data[stock_name] = df["CLOSE_PRICE"].values
# Combine into one DataFrame
prices = pd.DataFrame(data)
```

Task 4 - a

I have picked SBI Nifty Consumption ETF (India) The List of Top 30 stocks is:

```
In [22]: for f in csv_files:
    # Extract stock name from file name (without .csv)
    stock_name = os.path.splitext(os.path.basename(f))[0]
    print(stock_name)
```

ADANIPOWER
APOLLOHOSP
ASIANPAINT
BAJAJ-AUTO
BHARTIARTL
BRITANNIA
COLPAL
DLF
DMART
EICHERMOT
ETERNAL
GODREJCP
HAVELLS
HEROMOTOCO
HINDUNILVR
INDHOTEL
INDIGO
ITC
M&M
MARUTI
MAXHEALTH
NAUKRI
NESTLEIND
TATACONSUM
TATAPOWER
TITAN
TRENT
TVSMOTOR
UNITDSPR
VBL

Task 4 - b

```
In [23]: # Print the stock price data (Task 4 - b)
print(prices)
```

	ADANIPOWER	APOLLOHOSP	ASIANPAINT	BAJAJ-AUTO	BHARTIARTL	BRITANNIA	\
0	518.25	6780.95	2467.9	7961.5	1798.8	5391.3	
1	542.20	6905.00	2445.0	8018.0	1822.6	5416.2	
2	545.55	7005.50	2432.0	8247.5	1889.1	5454.6	
3	549.70	7073.50	2453.1	8141.0	1883.4	5393.2	
4	569.05	7142.50	2464.4	8261.5	1852.2	5435.5	
..	...	...	...	...	...	...	
115	643.65	7825.50	2537.3	9383.0	1893.7	6142.5	
116	637.35	7852.50	2480.9	9072.5	1939.5	6202.5	
117	609.70	7804.50	2579.9	9084.5	1895.3	6076.0	
118	624.65	7879.50	2555.9	9113.5	1913.0	6300.5	
119	709.40	7806.50	2481.7	8971.5	1962.4	6069.5	

	COLPAL	DLF	DMART	EICHERMOT	...	MAXHEALTH	NAUKRI	NESTLEIND	\
0	2491.65	623.60	4131.5	5529.0	...	1077.5	6776.0	2367.9	
1	2513.70	662.80	4212.7	5615.5	...	1084.4	6734.5	2383.3	
2	2551.60	656.25	4221.5	5678.5	...	1073.0	6939.0	2416.6	
3	2570.50	669.40	4357.1	5810.5	...	1128.7	6953.5	2399.4	
4	2544.00	675.90	4342.3	5784.0	...	1129.6	7085.5	2407.3	
..	...	...	...	...	...	...	...	...	
115	2398.90	751.10	4759.8	6874.5	...	1152.9	1349.5	1202.2	
116	2355.80	786.75	4696.2	6927.5	...	1158.7	1352.4	1204.2	
117	2423.30	755.55	4793.3	6580.5	...	1170.0	1347.0	1208.5	
118	2374.60	756.35	4589.7	6758.0	...	1179.3	1371.9	1220.8	
119	2337.90	777.70	4775.3	6973.5	...	1159.7	1376.1	1194.5	

	TATACONSUM	TATAPOWER	TITAN	TRENT	TVSMOTOR	UNITDSPR	VBL
0	1097.9	364.45	3234.9	4877.0	2586.5	1476.9	550.60
1	1103.4	378.50	3273.4	5035.5	2614.2	1501.7	549.20
2	1105.9	381.25	3272.1	5130.5	2679.3	1503.2	556.75
3	1120.2	381.70	3327.5	5360.0	2735.6	1517.8	546.80
4	1121.0	390.85	3336.1	5323.0	2738.5	1516.7	549.45
..	...	...	...	...	...	...	...
115	1083.5	384.65	3630.1	5218.5	3566.3	1304.4	474.50
116	1092.1	396.05	3556.8	5147.5	3499.9	1330.0	462.90
117	1072.4	385.80	3665.3	5528.5	3476.6	1312.1	469.65
118	1105.3	388.10	3584.2	5171.0	3512.1	1319.0	474.65
119	1126.8	396.30	3467.2	5080.0	3529.4	1328.2	474.45

[120 rows x 30 columns]

Task 4 - c

```
In [24]: # Compute the daily returns (Task 4 - c)
returns = prices.pct_change().dropna()
returns.head()
```

```
Out[24]:
```

	ADANIPOWER	APOLLOHOSP	ASIANPAINT	BAJAJ-AUTO	BHARTIARTL	BRITANNIA	COLPAL	DLF	DMART
1	0.046213	0.018294	-0.009279	0.007097	0.013231	0.004619	0.008850	0.062861	0.01965
2	0.006179	0.014555	-0.005317	0.028623	0.036486	0.007090	0.015077	-0.009882	0.00208
3	0.007607	0.009707	0.008676	-0.012913	-0.003017	-0.011257	0.007407	0.020038	0.03212
4	0.035201	0.009755	0.004606	0.014802	-0.016566	0.007843	-0.010309	0.009710	-0.00335
5	0.007556	-0.007560	-0.013310	-0.007263	0.015873	0.020017	0.045126	0.013464	0.02148

5 rows x 30 columns



Task 4 - d

```
In [25]: # Compute the covariance matrix (Task 4 - d)
cov_matrix = returns.cov()
cov_matrix.head()
```

Out[25]:

	ADANIPOWER	APOLLOHOSP	ASIANPAINT	BAJAJ-AUTO	BHARTIARTL	BRITANNIA	COLPAL	
ADANIPOWER	0.000553	0.000044	-0.000019	-0.000007	0.000029	-0.000009	0.000040	0.000
APOLLOHOSP	0.000044	0.000180	0.000025	0.000013	0.000018	0.000016	0.000027	0.000
ASIANPAINT	-0.000019	0.000025	0.000159	0.000015	-0.000027	-0.000002	0.000044	-0.000
BAJAJ-AUTO	-0.000007	0.000013	0.000015	0.000248	-0.000014	0.000026	0.000017	-0.000
BHARTIARTL	0.000029	0.000018	-0.000027	-0.000014	0.000159	0.000027	-0.000012	0.000

5 rows × 30 columns



Task 4 - e

In [26]:

```
# Compute PCA (Task 4 - e)
# Eigen-decomposition of covariance matrix
eigenvalues, eigenvectors = np.linalg.eigh(cov_matrix) # eigh for symmetric matrices

# Sort in descending order
idx = np.argsort(eigenvalues)[::-1]
eigenvalues = eigenvalues[idx]
eigenvectors = eigenvectors[:, idx]

# eigenvalues: variance explained by each principal component
# eigenvectors: loadings (which stock contributes to each PC)
print("Eigenvalues (variance explained):", eigenvalues)
print("Eigenvectors:", eigenvectors)
print("Eigenvectors shape:", eigenvectors.shape) # (30,30)
```

Eigenvalues (variance explained): [5.84894105e-03 2.50199986e-03 1.17781968e-03 9.12734327e-04
6.12763280e-04 5.13651996e-04 4.24281300e-04 3.75881605e-04
3.51235518e-04 2.83613724e-04 2.57984367e-04 2.17645546e-04
2.00746685e-04 1.80779919e-04 1.54210900e-04 1.35316993e-04
1.24350836e-04 1.19107821e-04 1.09353025e-04 1.03855147e-04
9.45920691e-05 8.30938146e-05 8.08105414e-05 6.57970318e-05
6.18699966e-05 5.63756670e-05 5.13460025e-05 4.61292182e-05
4.28033066e-05 3.60945497e-05]

Eigenvectors: [[-7.90785865e-02 -5.97976986e-02 4.65937544e-01 -6.73462795e-02
2.87986236e-01 -1.26412314e-01 2.89113005e-02 -5.74465093e-02
5.22536358e-01 -5.18273759e-01 -3.05897815e-02 -7.54988146e-03
1.42717544e-01 5.20472416e-02 -1.54401150e-01 -9.10331029e-02
-3.85753204e-02 -8.78118193e-02 4.64938851e-02 3.17553731e-02
4.55508386e-02 1.16298139e-01 4.34216168e-02 -1.09995264e-01
-6.90901456e-02 -4.95449917e-02 -2.75271028e-02 -2.66773642e-02
1.12052886e-01 6.65563597e-02]
[-9.52429633e-05 -5.12908798e-02 1.44786502e-01 -3.51940145e-02
1.23546291e-01 -2.98060619e-02 -1.24381445e-01 -8.08463319e-02
-2.40334559e-02 2.42471143e-01 -6.25083861e-02 2.61996870e-01
9.91059956e-02 2.05277501e-01 -4.12043102e-01 4.66293985e-01
-8.19143190e-02 -2.19263203e-01 1.83891341e-01 1.37349788e-02
2.89542944e-01 -1.52916624e-01 2.56536853e-01 6.30468523e-02
-1.91947060e-01 -1.70144495e-01 1.19398197e-03 -1.24081315e-01
2.79899081e-02 -1.34718391e-01]
[1.43911131e-02 -2.88922702e-02 -4.35537103e-02 3.45680591e-03
-2.03630865e-01 -1.24665557e-01 7.67988121e-02 -6.46727921e-02
2.23998191e-01 1.40267476e-02 -1.42135316e-01 1.49400582e-01
-7.17836658e-02 -1.35118415e-02 9.16657420e-02 3.86290389e-01
-4.85951605e-01 -1.35673807e-01 8.26632950e-02 3.26777686e-03
-4.39785323e-01 1.66846127e-01 -3.39886543e-01 -5.98802007e-02
9.19043523e-03 7.95982393e-02 1.10440774e-01 2.23229624e-01
-4.93475432e-02 -3.82972947e-02]
[2.72174233e-02 -3.51395753e-02 2.98840967e-02 2.66982384e-01
1.75069240e-02 -2.11081326e-01 -7.42210881e-02 -1.55237829e-01
1.70519690e-01 3.32850859e-01 4.26948974e-01 -1.40855905e-01
-1.05451849e-01 5.44820399e-02 1.37110911e-01 -9.89493355e-03
-1.89548854e-01 1.47851508e-01 1.26823049e-01 2.86365972e-01
2.95975195e-01 2.77638562e-01 7.28074682e-02 -2.63171448e-01
1.22236517e-01 1.02285002e-01 1.26725523e-01 -3.98623207e-02
1.95457911e-01 5.00667502e-03]
[-1.13379114e-02 -9.14326978e-02 6.11273733e-02 7.62774396e-02
1.74962899e-01 1.99551242e-01 -4.11286054e-02 4.01719686e-02
-1.76841308e-01 6.17298169e-02 -2.12700284e-01 3.46082140e-02
-5.13490857e-02 2.05942236e-01 -2.13682782e-02 -5.42243867e-02
-2.39695200e-01 2.61932894e-02 2.21443595e-02 -1.84971263e-01
-2.18492538e-01 1.85719060e-01 3.65389627e-01 -3.87126250e-04
4.59126903e-01 -3.09154741e-01 1.60789425e-01 -2.58711873e-04
2.51744548e-01 2.72144396e-01]
[3.33137334e-04 -8.16215261e-02 -2.42215174e-02 6.65078441e-02
3.13854177e-02 -9.26080955e-02 -5.56137574e-02 -2.29348165e-01
-1.94501414e-01 -4.52298717e-02 -7.95774015e-02 -6.99454194e-02
4.08125828e-01 5.05342188e-01 3.41026468e-01 7.59746915e-02
1.11451095e-01 -1.48734891e-01 -2.10699359e-01 3.44875306e-01
-5.57209170e-02 -9.06626013e-02 -1.02045714e-01 -1.01926748e-01
-1.21267232e-01 -1.40912788e-01 -5.13783500e-02 1.10154548e-01
-2.81672527e-02 2.18701025e-01]
[-4.78866732e-02 -6.16481086e-02 9.87975967e-02 6.36701708e-02
-3.03015610e-01 -1.92910173e-01 3.46379909e-01 -1.13721311e-01
-2.88244775e-02 7.47677492e-02 -1.80113992e-01 -1.26014647e-01
4.46639580e-02 1.36057908e-01 3.36329401e-02 -2.79436715e-01
-1.36175767e-02 -1.21260450e-01 -8.22130631e-02 -3.76097910e-01
2.51390073e-01 1.58413492e-01 -2.01555893e-01 2.22052981e-03
-9.02452137e-02 -1.99915635e-01 4.11832257e-01 -1.96507935e-01
-8.60812157e-02 -8.90021384e-02]
[-1.18269626e-02 -7.91145546e-02 4.32318108e-01 -1.20751325e-01
-3.22558550e-02 1.49710488e-01 -2.18937577e-01 2.15879355e-01
-1.51268461e-01 2.30687697e-01 1.53404891e-01 -3.19921784e-01
-7.15105559e-02 -7.04590087e-02 -1.68078890e-01 4.26556491e-02
2.48459061e-01 -3.91869901e-01 3.19293426e-02 9.78231643e-02
-1.20315566e-01 2.34807208e-01 -3.06453278e-01 2.93528078e-03
1.10857822e-01 -5.55294473e-02 4.15750687e-02 8.56719440e-02
-9.82600764e-02 9.21318676e-02]
[5.90606207e-03 -6.98399408e-02 1.90750519e-01 1.41727720e-01
-1.33917135e-01 -4.25147224e-03 3.40711306e-01 3.04224851e-01
1.21134516e-01 1.23606174e-01 -1.70788809e-01 4.19124089e-01]

-3.27180776e-01 2.03197190e-01 7.02789850e-03 -4.85869584e-02
1.97759312e-01 -2.17070902e-02 -3.10585945e-01 1.98959049e-01
1.40846273e-01 5.91415751e-02 3.48155823e-02 -1.06310560e-01
1.50857154e-01 1.05933220e-01 -2.10147224e-01 1.82567686e-01
-5.15686166e-02 2.97363069e-02]
[-9.80656394e-03 -4.68119127e-02 8.77134286e-02 1.79568089e-01
2.67604387e-01 -6.88518727e-03 -5.74690126e-03 5.27412435e-02
-7.04514763e-02 -6.38240101e-02 -4.84547870e-02 6.58902802e-02
2.15263991e-01 1.30202776e-01 1.22291579e-01 2.93202084e-01
7.21422073e-02 1.78781869e-01 -2.35369531e-02 -4.61746469e-02
6.56290155e-02 3.16356331e-01 -1.44436994e-01 3.61504411e-01
3.06790959e-01 2.12180246e-01 -4.99089913e-02 -3.39024412e-01
-2.47627155e-01 -2.99468302e-01]
[5.35309188e-02 -9.17043397e-03 7.46556789e-02 -1.78809136e-02
-2.82294795e-02 7.26509338e-01 3.11716558e-01 -5.17867092e-01
9.37395978e-02 -2.37473959e-02 7.11874435e-02 -1.23214384e-01
-9.63355993e-02 -2.61172390e-03 -9.10911059e-03 1.17274015e-01
-1.23093550e-02 4.27287034e-03 -1.19326492e-01 7.25647876e-02
8.20074518e-02 3.01171180e-02 -1.67705101e-02 3.16813258e-02
-6.35473670e-03 3.30142044e-02 -5.59801182e-03 5.51311292e-02
1.57219936e-02 -1.06897721e-01]
[-4.94682210e-03 -8.01263333e-02 8.81993761e-02 -9.32128593e-02
-1.22187831e-01 -2.26856594e-01 9.34141566e-02 -3.25670955e-01
-2.36737077e-01 -1.33290477e-01 -1.26505028e-01 -1.46748630e-02
-5.42885027e-02 -2.07171416e-01 -1.08905474e-02 1.40582585e-01
2.60786040e-01 1.64298151e-01 2.30178873e-01 -4.49425481e-03
-7.45672900e-02 1.08726807e-01 1.10804957e-01 -4.71777947e-01
2.49213954e-01 -2.11681271e-01 -2.20040804e-01 4.03685755e-02
-2.36772947e-01 -1.63895456e-01]
[-1.13712335e-02 -4.46427354e-02 2.00859737e-01 7.29254031e-02
4.87381694e-02 5.47513132e-02 1.14323297e-01 2.53960670e-02
2.64329673e-02 2.09566194e-01 -4.72648663e-02 -1.58932080e-01
-4.85995057e-02 2.43814658e-01 2.68573541e-01 -1.95690672e-01
-1.38365152e-01 -2.34567350e-01 3.56565814e-01 -2.28552931e-01
-1.74216451e-01 -2.76037920e-01 6.89711066e-02 -1.23438490e-01
-2.49610300e-02 2.46174487e-01 -4.13161086e-01 -2.05482133e-01
2.60949562e-02 -1.84030716e-01]
[-2.21289371e-02 -9.00274891e-02 1.17791069e-01 3.65852893e-01
1.07099337e-01 -2.05968949e-01 7.36693071e-02 -9.37803803e-02
8.56062116e-02 5.07115681e-02 -9.29567428e-02 -3.14348441e-01
-2.25352542e-01 -1.81323940e-01 -1.26214095e-01 2.29160120e-01
-2.69597914e-02 7.22885566e-02 -2.79248422e-01 5.58311192e-02
-7.34778837e-02 -5.19745641e-01 -1.44402898e-01 9.10106915e-02
1.64623580e-01 -1.61137710e-01 -1.40120982e-02 -2.06842164e-01
2.44924249e-02 1.33188902e-01]
[-1.62444777e-02 -3.77245238e-02 1.53460284e-02 5.84686634e-02
-1.92984429e-01 -2.03085975e-01 1.59576360e-01 -2.30531853e-01
-1.36061117e-01 -9.75416482e-02 1.62600007e-02 -1.33208140e-02
1.44757516e-04 -1.98867878e-01 -3.53624316e-02 2.74860595e-02
9.65105375e-02 -3.46564077e-01 -7.63926672e-02 9.08835935e-02
-1.73793158e-01 1.55571440e-01 4.19809612e-01 2.02770284e-01
-1.62052280e-02 4.76539590e-01 6.83247013e-02 -1.84665990e-01
1.24083896e-02 2.93229898e-01]
[-9.44064968e-03 -5.04411095e-02 3.27019433e-01 -9.93958398e-02
-2.20963476e-01 -4.83773627e-02 -1.34995401e-01 -3.64490317e-02
4.04928841e-02 1.28817328e-01 2.20719974e-01 -2.70252275e-02
-3.52716093e-02 2.04812257e-01 -1.53322697e-02 4.92690234e-02
9.46645047e-02 4.86531387e-01 -3.09537302e-01 -1.94676953e-01
-4.17303223e-01 6.43246895e-02 1.88460028e-01 2.46528859e-02
-2.99535979e-01 1.43477233e-02 9.41607371e-03 -6.62305051e-02
2.19922026e-02 -8.72595599e-02]
[-4.87908675e-02 -6.42960573e-02 2.75824823e-01 -5.71203836e-02
-1.87049075e-01 4.22588888e-02 -1.03434579e-01 -3.80131307e-02
2.71998024e-02 2.01832233e-02 2.40566722e-01 3.60349191e-01
1.65113761e-01 -4.05175752e-01 5.17837268e-01 3.70664225e-02
-6.68567151e-02 -1.23292006e-01 -1.07691975e-01 -1.32698831e-01
1.29165251e-01 -1.13300376e-01 -2.92907195e-03 1.42821643e-01
1.23058850e-01 -2.60758817e-01 -1.12422598e-01 -6.38022561e-02
1.09832994e-01 6.84114262e-02]
[-2.27567475e-02 -1.39801691e-02 8.45943887e-02 -5.58339911e-02
-5.64526042e-02 -1.48085180e-01 -4.64264005e-02 -1.00568734e-01
-1.15817252e-01 -7.94079300e-04 -7.86698571e-02 -3.73515133e-02
2.02998226e-01 -2.41354121e-02 -1.09687698e-01 -9.60826002e-02
6.76585972e-02 -7.23470804e-02 -1.71687638e-01 1.87056971e-02
-2.51419888e-02 -1.55252030e-01 -2.63460267e-02 5.99524272e-02

2.82960810e-01 2.13772404e-01 1.05045766e-01 3.30884722e-01
5.24765902e-01 -5.26544357e-01]
[-1.51015614e-02 -8.56777431e-02 2.79452685e-02 3.65127983e-01
9.78466885e-02 1.46681402e-01 7.82320152e-02 -7.35466625e-02
-3.63986140e-02 1.75258233e-01 1.35215992e-01 3.72521914e-01
4.28001654e-01 -1.31184414e-01 -3.17751060e-01 -1.52135982e-01
1.13439402e-01 7.85844720e-02 7.45906468e-02 -1.45932167e-01
-1.90299191e-01 -1.01604193e-01 -2.31973781e-01 -2.84896581e-01
6.82051679e-03 1.68018803e-01 9.00514575e-02 -3.69084397e-02
-1.80333869e-02 1.91359349e-01]
[-3.47662617e-03 -4.90852095e-02 1.32718603e-01 1.98192380e-01
3.72474516e-01 6.78140969e-02 1.38503639e-01 1.64865406e-01
-2.08892123e-01 1.38062105e-02 -1.28607975e-01 8.83984919e-02
-1.26910731e-01 -2.32689041e-01 2.57346566e-01 9.10526662e-02
8.37644807e-02 -7.87079405e-02 5.34317485e-02 1.92166886e-01
-1.51726226e-01 3.02074401e-02 1.50312226e-01 -1.40638385e-01
-4.22708837e-01 -9.73795507e-02 3.69943800e-01 -3.53634693e-02
8.96326537e-02 -2.90561077e-01]
[2.02583088e-02 -5.08445290e-02 -1.08701280e-02 6.46235936e-02
-3.97788043e-02 1.29810839e-01 -5.89427318e-01 -1.90800682e-01
2.35192159e-01 1.72079976e-01 -5.55829824e-01 2.74908233e-02
-5.83408239e-02 -1.40322972e-01 1.05753597e-01 -1.14398462e-01
6.17010158e-02 -2.04964987e-02 -1.83040348e-01 -2.93871225e-02
1.01152715e-01 8.43332699e-02 1.43461984e-02 -1.83506448e-01
-4.81509588e-02 1.89679065e-01 3.34026520e-02 -1.19368191e-01
-6.99521932e-02 -8.92183694e-03]
[9.88203873e-01 -5.76916447e-03 8.06580133e-02 -3.24427011e-02
1.49408556e-02 -5.75530757e-02 3.43763786e-02 3.11871050e-02
2.21861960e-02 -1.32646533e-02 -2.15124160e-02 -6.94808197e-03
6.85924904e-02 -2.73122527e-02 6.71190942e-03 -1.19515059e-02
2.54650675e-03 -7.10158791e-03 -7.37396943e-03 -3.26736891e-02
3.84408874e-03 -1.82558414e-02 8.85836514e-03 2.40201563e-03
1.50181674e-02 -1.17563154e-02 1.88313465e-02 -2.13958349e-02
1.24013893e-02 2.03100678e-02]
[6.66907790e-03 -9.49787683e-01 -2.19189723e-01 -1.56956420e-01
8.24934066e-03 2.58949082e-02 1.87729140e-02 7.26292957e-02
5.98071112e-02 -1.92708865e-02 5.23793410e-02 -3.09707941e-02
-1.08170214e-02 -2.92627746e-02 -1.86479176e-02 -2.99702429e-02
6.93821558e-04 2.67913006e-02 2.08436812e-02 2.38528507e-02
1.89288359e-02 -8.75798179e-03 -6.25661876e-03 4.46410881e-02
-4.14539556e-02 2.17226542e-02 -1.64895807e-02 -1.96006468e-02
2.63449318e-02 -3.48421084e-02]
[-1.28255958e-03 -4.78010919e-02 2.27555497e-01 -6.09662088e-02
3.99572476e-02 -6.31578624e-02 2.18708087e-02 -1.41180685e-01
-3.49609364e-01 -2.91644639e-02 -2.38307982e-01 7.50337147e-02
-1.80457139e-01 -2.61540130e-02 2.63317626e-03 3.96359709e-02
-4.02989052e-02 2.98684298e-01 2.32755874e-01 -1.62358052e-02
1.89069877e-01 1.04708737e-01 -3.38278173e-01 1.68193496e-01
-2.29000654e-01 1.55559503e-01 -1.71896626e-01 2.56598087e-02
3.99630890e-01 3.07627903e-01]
[-2.74313515e-02 -7.05144387e-02 3.09721322e-01 -8.77854057e-02
-5.23079171e-02 4.07822900e-02 -4.57952372e-02 2.71505037e-02
-3.44433411e-02 -4.09254411e-02 -4.44829724e-02 -1.77052831e-02
1.47227863e-02 6.09003393e-02 1.38007360e-01 -7.15772006e-03
-7.22906200e-02 2.26308959e-01 2.53191285e-01 5.19640141e-02
1.45027408e-01 -3.68949420e-01 1.26600568e-01 7.80850897e-02
1.54584095e-01 2.75146670e-01 4.35173203e-01 3.14737854e-01
-3.86507772e-01 1.23598029e-01]
[6.87653920e-03 6.70381438e-03 9.43535750e-03 1.91454971e-01
-2.34416771e-01 5.02279292e-03 3.89263593e-02 -7.52581095e-02
2.53797143e-01 1.51398154e-01 -1.77678299e-01 1.36218781e-02
1.05158819e-01 -5.94716293e-02 -2.97674416e-02 -2.78342857e-01
1.84691223e-01 1.33719042e-01 3.81704122e-01 3.91221058e-01
-2.29117429e-01 5.11809085e-02 3.48638658e-02 4.42225870e-01
-2.15298823e-03 -2.80251903e-01 -4.18313987e-02 1.98126335e-02
1.23755465e-02 -7.40555457e-02]
[3.03770927e-02 -5.25460304e-02 1.08382028e-02 4.87680347e-01
-4.95606679e-01 2.32556795e-01 -1.76715506e-01 2.89262623e-01
-1.16740035e-01 -4.81515432e-01 2.43338478e-02 -1.45768154e-01
2.52539645e-02 5.22305429e-02 9.31168001e-03 1.14194640e-01
-2.64223178e-02 2.66661385e-02 1.02378246e-01 -6.48619793e-03
7.43485531e-02 1.42524487e-02 2.67498691e-02 -1.32170535e-01
-3.63587644e-02 2.85821031e-02 -2.07104514e-02 -3.88966438e-02
1.29885153e-01 -5.60264473e-02]
[7.31060315e-03 -8.01098619e-02 3.30115576e-02 3.82569902e-01

```

1.57967048e-01 -9.45745406e-02 -1.19852171e-02 -6.83969363e-02
-2.62665064e-02 9.42505582e-02 -1.26267709e-02 -1.69616312e-01
9.47053313e-02 -1.29628828e-01 -2.89967682e-02 -6.72068336e-04
-9.95527807e-02 -4.69917358e-02 -8.25883300e-02 -3.09317170e-01
1.29501491e-01 1.83953519e-01 1.82513759e-01 1.92578639e-01
-2.06837889e-01 -6.42301481e-02 -2.75028308e-01 5.85751394e-01
-2.02335690e-01 -5.18155116e-03]
[-5.80509527e-03 -2.47339680e-02 1.83947629e-01 -5.97010109e-02
-4.74247337e-02 2.18176288e-02 -2.95272730e-03 3.08211198e-02
-3.14219324e-01 -2.71886039e-02 -4.42351339e-02 9.70471294e-03
1.15782230e-01 -1.39241742e-01 -2.11555498e-01 -3.32353158e-01
-5.93568399e-01 5.87201564e-02 -2.26398202e-01 3.62266085e-01
1.82397205e-02 2.78081810e-02 1.48167009e-02 -2.97311735e-02
-4.20416838e-02 -2.74167452e-02 -1.65310939e-01 -1.46090746e-01
-2.36985379e-01 -1.44611409e-01]
[6.55153946e-02 -5.58254190e-02 -1.40302124e-02 1.88617356e-01
9.95707562e-02 -8.25617011e-02 -3.04640008e-01 -3.34964080e-01
-1.49280546e-01 -2.41393480e-01 2.61899532e-01 3.29539212e-01
-4.63012709e-01 2.26951855e-01 -6.09026861e-02 -2.48484800e-01
2.58971550e-02 -1.94133452e-01 3.75788555e-02 -4.94196014e-02
-9.47239812e-02 -6.34755330e-02 -1.32910716e-01 1.54496388e-01
1.64065378e-02 -7.19449888e-02 7.71038836e-02 2.49070867e-02
-1.33496416e-01 -1.39337538e-01]]
Eigenvectors shape: (30, 30)

```

Task 4 - f

```

In [27]: # Perform SVD for stock returns (Task 4 - f)
X = returns - returns.mean()
U, s, VT = np.linalg.svd(X, full_matrices=False)

```

```

In [28]: # Present the result
print("Stock Returns Matrix Dimension:")
print(returns.shape)
print("\nDimension of Matrix U:")
print(U.shape)
print("\nSingular values:")
print(s)
print("\nDimension of Matrix V^T:")
print(VT.shape)

```

Stock Returns Matrix Dimension:
(119, 30)

Dimension of Matrix U:
(119, 30)

Singular values:
[0.83076774 0.54335622 0.37280387 0.32818082 0.26889787 0.24619288
0.22375253 0.21060396 0.20358239 0.18293829 0.1744768 0.16025659
0.15390942 0.14605489 0.13489583 0.1263622 0.1211338 0.11855262
0.11359426 0.11070188 0.10564972 0.09902055 0.09765062 0.08811385
0.0854439 0.08156181 0.07783848 0.07377837 0.07106891 0.06526222]

Dimension of Matrix V^T:
(30, 30)

Explanation

The stock prices can't be compared, so we calculate the **returns** using the prices, which *can* be compared and analyzed.

The **covariance matrix** shows the relationship between every pair of stocks. It ranges from **-1 to 1** — the higher the value, the stronger the relationship, and vice versa.

Two stocks having a strong relationship tend to move in a similar fashion.

PCA (Principal Component Analysis) is a *statistical technique*, while **SVD (Singular Value Decomposition)** is a *pure linear algebra operation*.

The eigenvalues start from **0.0058** and quickly drop — meaning the first few components explain most of the variance, forming the classic *"elbow" shape* you'd expect in PCA.

Each column in your eigenvector matrix represents a **principal component**, i.e., a direction in feature space where variance is maximized.

The first few singular values (**0.83, 0.54, 0.37...**) are significantly larger than the rest, indicating that the dataset has a strong internal structure dominated by a few key patterns.

The sharp decline from **0.83 to 0.37** shows that the first two or three components capture most of the systematic variation in the data — possibly reflecting broad **market behavior** or **sector-level trends** influencing multiple stocks simultaneously.

Together, these top components could explain roughly **65–70%** of the total variance, meaning a large portion of the dataset's complexity can be summarized using only a small subset of components.

After the **5th or 6th component**, the singular values level off around **0.2–0.06**, suggesting diminishing returns — additional components add very little new information.

This flattening pattern is typical in financial data, where a few dominant **market factors** (like macroeconomic sentiment or liquidity conditions) explain most price movements, while the remaining variance comes from **stock-specific noise**.

Beyond the **10th component**, the singular values become small and almost uniform, indicating that the remaining components capture **residual randomness** rather than meaningful structure.

In practice, these lower components can be ignored during dimensionality reduction without losing significant explanatory power.

The ratio between the first and last singular values ($0.83 / 0.065 \approx 12.7$) is quite steep, confirming that the dataset is effectively **low-rank**.

This steep drop reflects a clear separation between dominant and weak components — a strong signal that the data is highly **compressible into a smaller subspace**.

This also implies that the dataset's variability is not spread evenly but concentrated in just a few dominant directions. In simpler terms, while the dataset may contain many variables or stocks, most of them behave in correlated patterns driven by a few underlying forces.

From a practical standpoint, this means that **dimensionality reduction using PCA or SVD** would be very efficient here.

Retaining just the **top five or six principal components** would likely preserve most of the variance, simplifying analysis and reducing computational load without sacrificing interpretive accuracy.

Such compression is extremely valuable in **quantitative finance**, where models benefit from focusing on dominant market signals rather than noise.

Overall, the **singular value spectrum** clearly demonstrates a market structure governed by a few powerful, shared movements